FINAL PROJECT / 2026

口罩佩戴检测系统

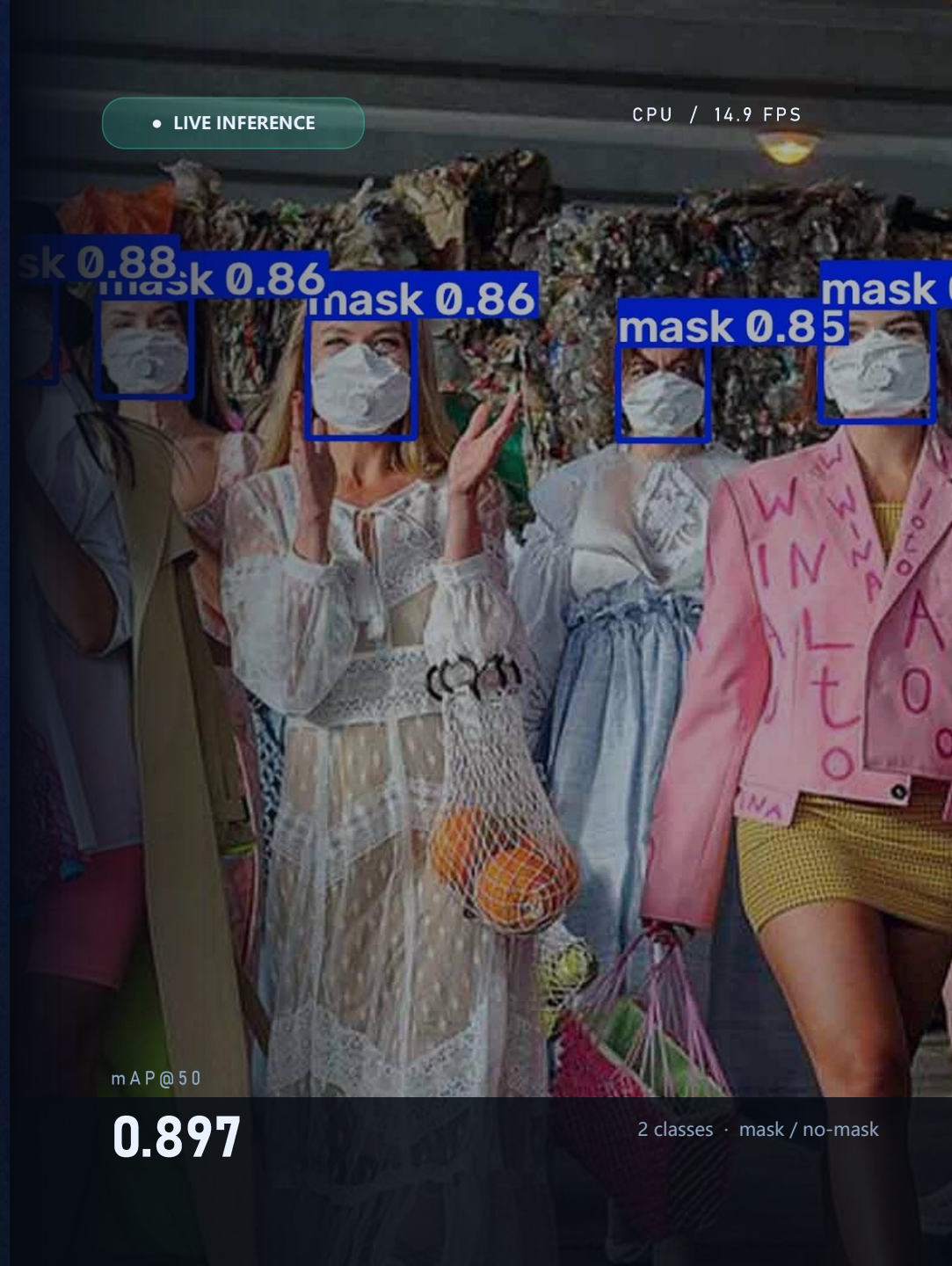
基于 YOLOv8 的目标检测与部署

YOLOv8n | ONNX Runtime | Streamlit

汇报人 刘漠荣 · 小组成员 姜瑞洋

202411009111 · 人工智能软件开发

2026 年 10 月



任务说明：检测每张图中的口罩目标

输入一张图片，输出每个目标的边界框、类别和置信度。



一张图里可能有多个目标，也可能没有目标

任务定义

$$f(I) \rightarrow \{ (x_1, y_1, x_2, y_2, c, p) \}$$

N 个目标 · 2 个类别 · 一次前向推理

WHERE

边界框

(x_1, y_1) 左上角
(x_2, y_2) 右下角

WHAT

类别 + 置信度

mask / no-mask
$p \in [0, 1]$

难点

多目标 | 小目标 | 遮挡 / 侧脸 | 类别不平衡

评价指标

mAP@50：IoU=0.5 时的平均精度；mAP@50-95：IoU 从 0.5 到 0.95 共 10 档取平均，更严格。

项目流程：数据、训练、调参、部署、网页

数据集 1,356 张，训练 100 轮，调参 3 组，最终部署到 Streamlit。

01

数据准备

`step2_dataset/prepare_dataset.py`

图片-标签配对
8:1:1 划分 · data.yaml

›

02

本地训练

`step3_train/train.py`

COCO 预训练权重
CPU · 100 epochs

›

03

GPU 调参

`step4_tuning/colab_tuning.py`

T4 · 3 组控制变量
50 epochs / 组

›

04

导出部署

`step5_deploy/deploy_and_evaluate.py`

.pt → .onnx
CPU 推理与测速

›

05

网页演示

`step6_web_demo/app.py`

Streamlit 上传
实时 conf / IoU

images

1,356 张

1084 / 135 / 137

local mAP@50

0.893

100 epochs · CPU

tuned mAP@50

0.897

50 epochs · T4

CPU throughput

14.9 FPS

67 ms / image

项目文件按阶段拆分

固定随机种子 42；所有路径相对项目根目录；每个阶段可单独运行。

7 scripts / 1,000+ lines

数据准备：图片与标签配对，按 8:1:1 划分

YOLO 按同名文件配对；固定 seed=42，保证每次划分结果一致。

step2_dataset/prepare_dataset.py

PYTHON

```python
image_files = [f for f in os.listdir(SRC)
               if f.lower().endswith((".jpg", ".jpeg", ".png"))]

for img in image_files:
    base  = os.path.splitext(img)[0]   # 0.jpg → 0
    label = base + ".txt"              # → 0.txt
    if os.path.exists(join(SRC, label)):
        pairs.append((img, label))

random.seed(42)                    # 固定划分
random.shuffle(pairs)
n_train = int(len(pairs) * 0.8)
train = pairs[:n_train]
val   = pairs[n_train:n_train + val_count]
```

划分结果

1,356

images · Kaggle Face Mask Detection



train 1,084　　val 135　　test 137

YOLO 标签格式

cls	cx	cy	w	h
0	0.512	0.438	0.296	0.407

全部归一化到 0–1；0=mask，1=no-mask

固定种子与扩展名过滤

seed=42 让每次划分一致；别忘了 jpeg，否则统计会少一批图。

YOLOv8n 的结构与迁移学习

Backbone 提取特征，Neck 融合多尺度，Head 输出框和类别；nano 版本用于 CPU 推理。

ONE-STAGE DETECTOR

单阶段检测流程



3.0 M
参数量

8.7 GFLOPs
640 输入

一次前向输出所有框和类别

n 为最小规格，s / m / l / x 依次增大

选择 nano 的主要原因是 CPU 推理速度

TRANSFER LEARNING

使用 COCO 预训练权重

COCO 80 类 → **mask / no-mask**

加载后自动把检测头从 80 类改为 2 类。

- 通用边缘 / 纹理特征
- 小数据也能快速收敛
- 训练成本显著下降

```
model = YOLO("yolov8n.pt")
Overriding nc=80 → nc=2
```

训练脚本：主要参数与输出

YOLOv8 已封装训练循环、数据加载和验证；脚本主要设置训练参数。

step3_train/train.py

PYTHON

```python
model = YOLO(WEIGHTS)          # COCO 预训练权重

results = model.train(
    data=DATA_YAML,            # 2 类: mask / no-mask
    epochs=100,
    batch=16,                  # CPU 内存的折中
    imgsz=640,                 # 统一输入尺寸
    project="runs/detect",
    name="train",
)
```

OVERRIDE nc=80 → nc=2

data DATA_YAML

唯一数据入口，告诉模型去哪里找图

epochs 100

100 轮；best.pt 按验证集表现选

batch 16

每批 16 张；内存不足就降到 8 / 4

imgsz 640

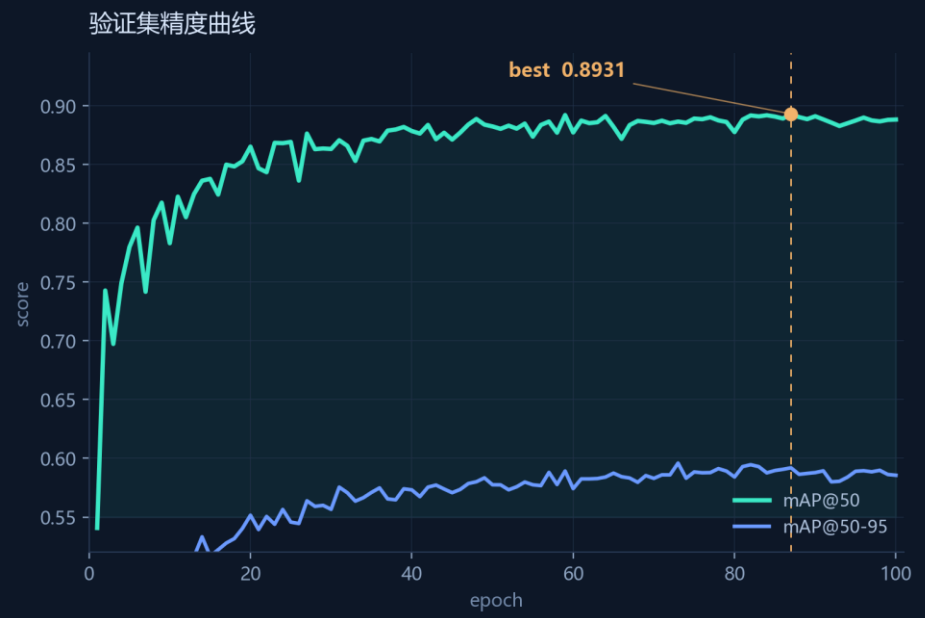
统一缩放；YOLOv8 的默认工作尺寸

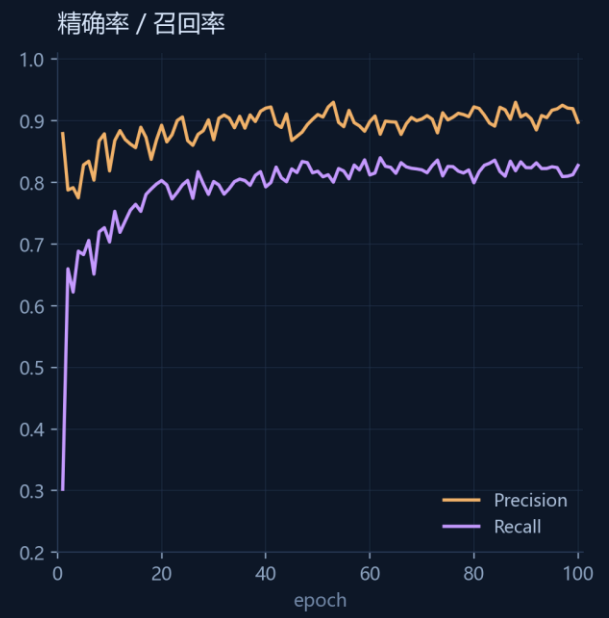
本地 CPU（R7 8845HS） **100 epochs** **6 h 08 min** ≈ 220 s / epoch 输出：best.pt · results.png · confusion_matrix.png

训练结果：mAP@50 0.8931，第 87 轮最佳

验证集 mAP 在 50 轮后基本稳定；best.pt 按验证集指标选择。

验证集 / 100 epochs





mAP@50 与 mAP@50-95 同图对比；右侧是 Precision / Recall。

mAP@50

0.8931

best @ 87

mAP@50-95

0.5959

best @ 73

Precision / Recall

0.903 / 0.834

训练耗时

6.14 h

CPU 100 epochs

第 50 轮后 mAP 基本稳定，后 50 轮提升约 1.1 个百分点。

调参设计：三组控制变量实验

三组实验都在 Colab T4 上训练 50 轮；只改学习率或 Mosaic，其余参数固定。

实验矩阵 3 × 50 epochs · Tesla T4

实验	学习率	优化器	Mosaic	
01 Baseline	0.01	SGD	开启	基线
02 lr 10×↓	0.001 ← 只改这里	SGD	开启	
03 no mosaic	0.01	SGD	关闭 ← 只改这里	

数据、模型、batch、imgsz 全部固定

step4_tuning/colab_tuning.py — PYTHON

```python
model.train(
    data=DATA_YAML, # Drive
    epochs=50, imgsz=640,
    lr0=0.01,          # ← 变量 1
    optimizer="SGD",
    mosaic=1.0,          # ← 变量 2
    device=0,
    project="/content/runs/tuning",
    name="exp1_lr001_sgd",
)
```

Colab 的 /content 会随断线清空 → 数据集放 Google Drive 挂载；结果先复制到 Drive 再下载。

调参结果：基线 0.8973，三组差距 1.2 个百分点

三组 mAP@50 的极差为 1.20 个百分点，差异不大。

mAP@50 对比

mAP@50，取每个实验的最高值



从 results.csv 取最佳轮次

```
best = max(rows, key=lambda r: float(r["metrics/mAP50(B)"]))
```

学习率 10×↓

−1.20 pt

50 轮内收敛不足，属于欠拟合

Mosaic 关闭

−0.74 pt

场景单一（正脸近照），增强收益有限

三组极差

1.20 pt

继续调参的空间有限

结论：三组差距小于 1.2 个百分点，后续提升应优先考虑数据。

模型导出与部署：ONNX 格式与 CPU 测速

ONNX 模型不再依赖 PyTorch；CPU 上 67 ms 一张，适用于静态图片。

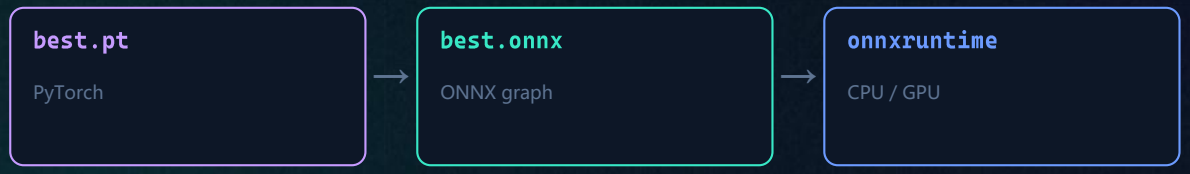
step5_deploy/deploy_and_evaluate.py

PYTHON

```python
from ultralytics import YOLO

model = YOLO("step4_tuning/best.pt")
onnx_path = model.export(
    format="onnx",    # 通用推理格式
    imgsz=640,
)
```

转换链路



CPU · R7 8845HS

14.9 FPS

67 ms / image

5 / 5 测试图检出成功



6.2 MB

ONNX 模型体积

14.9 FPS

静态图片可用

边界：不是实时视频

30 FPS 实时视频需要 33 ms / 帧；67 ms 只够做图片检测。上视频要 GPU、TensorRT 或 INT8 量化。

网页演示：Streamlit 推理流程

上传图片后，页面完成模型加载、推理、画框和结果统计。



一次交互 = 脚本从头重跑一次

推理流程

01 **选图** — 上传图片，或直接选择测试集图片

02 **加载模型** — cache_resource 缓存模型，并做一次预热

03 **执行推理** — model(image, conf, iou) 得到检测框

04 **展示结果** — plot 画框、统计数量、显示检测明细

两个实现细节

1 缓存 + 预热

首次加载约 6 s，之后单张约 80 ms；预热放在缓存函数里，只执行一次。

2 PIL 输入 / BGR→RGB

直接传 PIL 图片；plot 返回 BGR，显示前用 [..., ::-1] 转成 RGB。

启动 `streamlit run step6_web_demo/app.py`

检测结果示例

测试集图片未参与训练；框上标注类别和置信度。



街道人群 · 5 个目标全部检出



多人场景 · 8 mask + 1 no-mask



单人近照 · mask 0.94

测试图检出	演示页检测目标	单张推理	等效帧率
5 / 5	9	74 ms	13.5 FPS

演示页支持实时切换置信度 / IoU：调高更严格、误检少；调低更敏感、漏检少。

项目总结

完成数据准备、训练、调参、ONNX 导出和 Streamlit 演示。

01 数据与训练

0.8931

mAP@50 · 本地 CPU

- 1,356 张图，8:1:1 固定种子划分
- COCO 预训练 + 2 类检测头
- 100 epochs，6.14 小时跑完

02 调参与评估

0.8973

mAP@50 · T4 50 epochs

- 3 组控制变量实验
- 学习率与 Mosaic 的边际影响
- 结论：优先补充数据

03 部署与交付

74 ms

Streamlit 单张推理

- ONNX 6.2 MB · CPU 14.9 FPS
- 缓存 + 预热，6 s → 80 ms
- 可上传图片，也可选测试集

交付物

7 个脚本　3 组实验　1 个 ONNX　1 个网页　11 页报告

打开 run_demo.bat 即可演示

代码、报告和演示页面均已整理。

固定 seed，结果可复现

不足与改进方向

三组调参实验差距很小；后续工作优先补数据，再做量化。

当前不足

数据场景单一

以正脸、近照为主；侧脸、遮挡、暗光样本不足，泛化能力未验证。

速度还不到实时

CPU 67 ms / 张 ≈ 14.9 FPS；实时 30 FPS 视频还差一截。

只做静态图片

没有视频流、跟踪、计数去重；网页演示是单图推理。

改进计划

01 先补数据 最高性价比

收集侧脸、遮挡、暗光、不同口罩类型；做硬例挖掘，再重训一版。

02 再压模型 工程优化

尝试 YOLOv8s/m 找精度-速度平衡；INT8 量化把 CPU 推理再压一半。

03 最后上视频 场景扩展

接 ONNX Runtime GPU / TensorRT，做视频流检测与简单跟踪。

优先补数据，再做 INT8 量化；继续调参的收益有限。

THANK YOU / Q&A

谢谢聆听

恳请老师批评指正

刘溧荣 · 姜瑞洋

202411009111 · 人工智能软件开发 · 2026 年 10 月

0.897 mAP@50　　**74 ms** WEB INFERENCE

